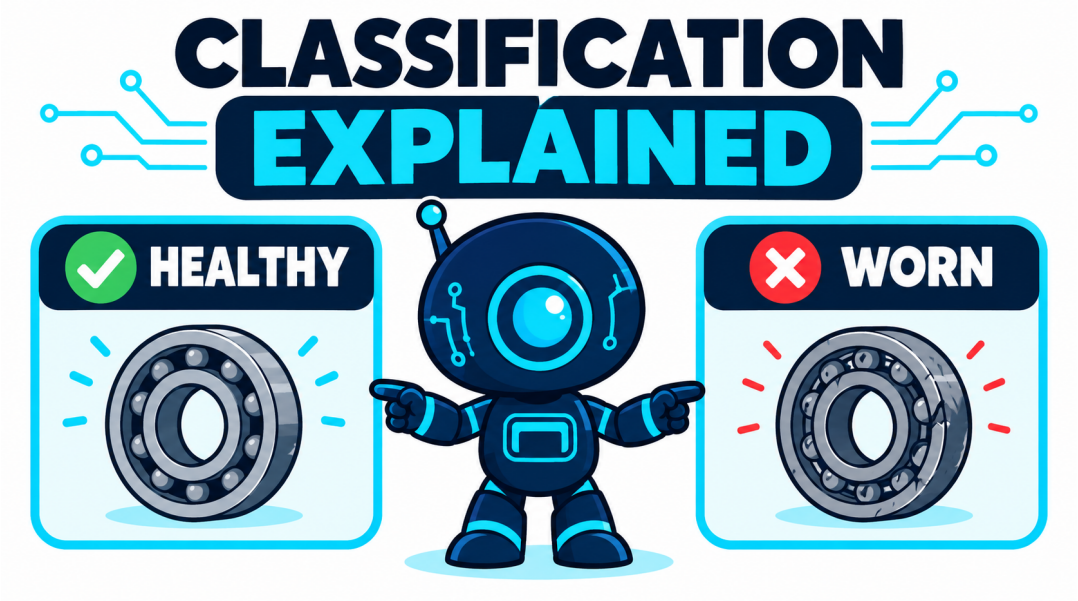
# AI MECH LAB

AI Course — Beginner Track




Module 4, Topic 4.3

## Classification

# Lesson

You've now seen the first branch of supervised learning: regression, where the target is a number on a continuous scale. Classification is the other branch — and the difference comes down to one thing. Instead of predicting a number, you're predicting which category something belongs to. The answer isn't "how much" anymore. It's "which one."

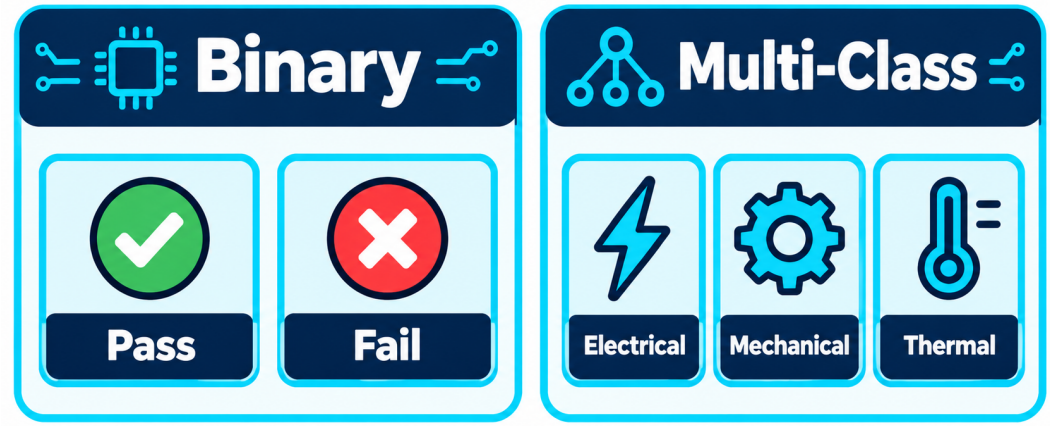
## What makes a target a classification problem

The test here mirrors the regression test, flipped: is the set of possible answers fixed and countable, with nothing in between? Predicting whether a bearing is "healthy" or "worn" is classification — there's no in-between value, no 1.5 that means "sort of worn." Predicting whether a welded joint passes or fails inspection is classification. Predicting which of three fault types a motor is showing — electrical, mechanical, or thermal — is also classification, just with more than two categories.

Compare that to predicting the exact vibration reading a bearing will produce — that's regression, because the answer can be any number along a range. Classification only makes sense when the answer has to land on one of a fixed set of labels.

## Binary vs. multi-class classification

Classification splits into two flavors depending on how many categories are possible.



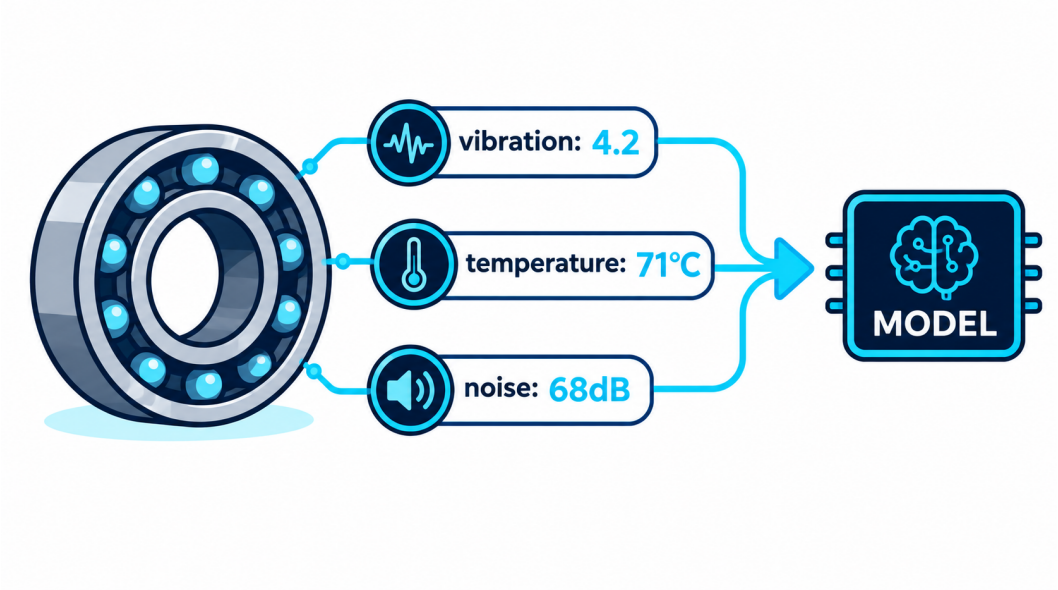

*Binary classification has two outcomes; multi-class has more than two.*

Binary classification has exactly two possible answers — pass/fail, healthy/faulty, normal/anomaly. A lot of quality-control and fault-detection systems in engineering are binary classifiers at heart: is this part within spec, yes or no?

Multi-class classification has more than two possible categories, but the answer still has to be exactly one of them. Sorting a motor fault into electrical, mechanical, or thermal is multi-class — three fixed categories, one correct answer per case.

## How a classification model actually decides

A classification model doesn't output a category directly at first — it outputs a probability for each possible category, and the category with the highest probability becomes the prediction.




*Sensor readings feeding into the model as input features.*

```
sensor_reading = {"vibration": 4.2, "temperature": 71, "noise_db": 68}
prediction = model.predict(sensor_reading)
# prediction: {"healthy": 0.08, "worn": 0.92}
# final output: "worn"
```



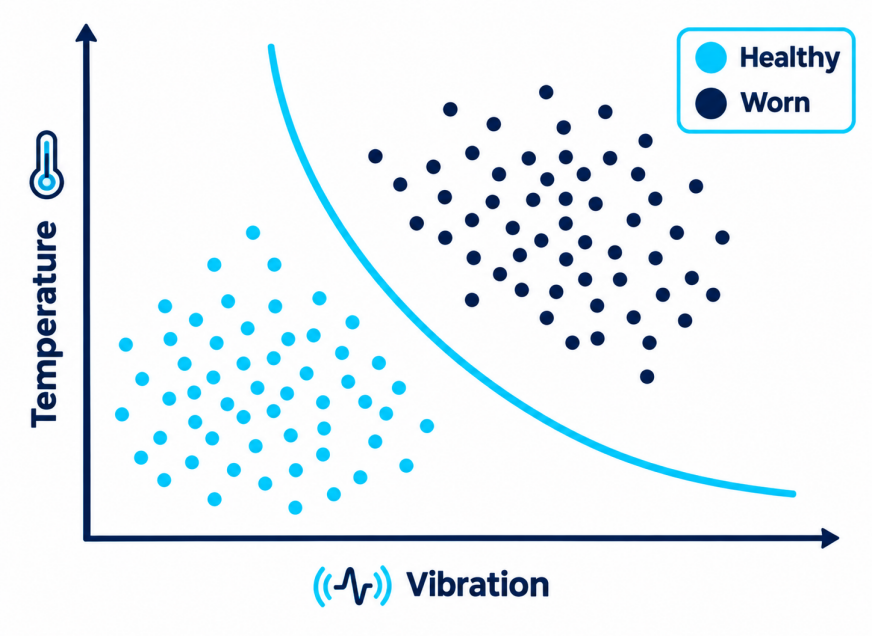

*The category with the highest probability becomes the final prediction.*

That "0.92" matters more than it looks like it does. It's not just saying "worn" — it's saying the model is 92% confident in that call. A prediction of 0.55 "worn" versus 0.45 "healthy" is technically still a "worn" prediction, but it's a much shakier one. Understanding that classification is built on probabilities, not certainties, is central to using these models responsibly — especially anywhere a wrong call has real

consequences, like flagging equipment for maintenance.

## Where the dividing line actually gets drawn

Picture two features — vibration and temperature — plotted against each other, with healthy readings clustered in one region and worn-bearing readings clustered in another. A classification model's job during training is to find the boundary that best separates those regions. New readings get classified based on which side of that boundary they fall on.

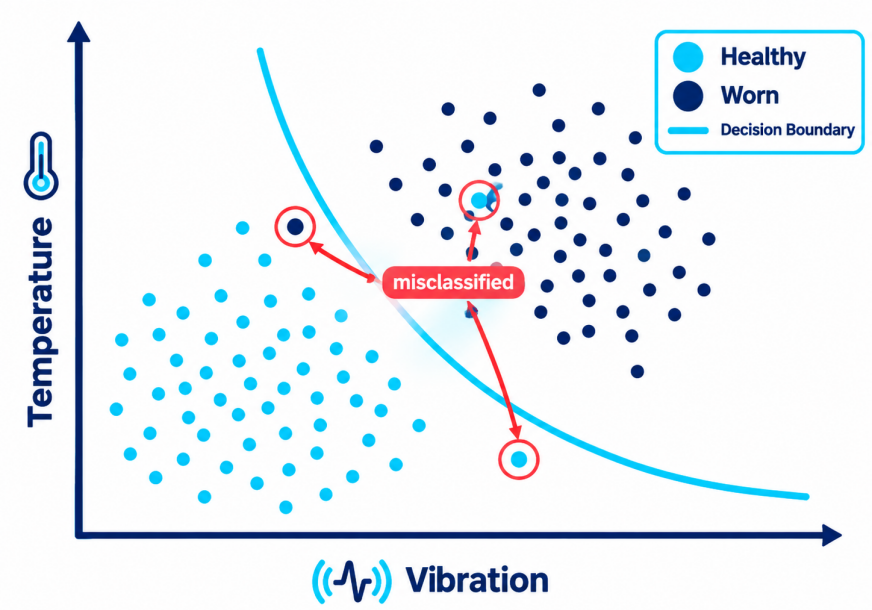



*The decision boundary separating healthy readings from worn readings.*

With two categories and two features, that boundary might be a fairly simple line or curve. With more categories or more features, the boundary gets more complex — but the underlying idea never changes: the model is learning where one category's territory ends and another's begins.

## Why the dividing line usually isn't perfect

Real sensor data doesn't sort itself cleanly. Some worn bearings will still read close to healthy values early in their decline, and some healthy bearings will have a rough reading on a bad day. A classification model trained on real data will almost always misclassify a few borderline cases — that's expected, not a failure.

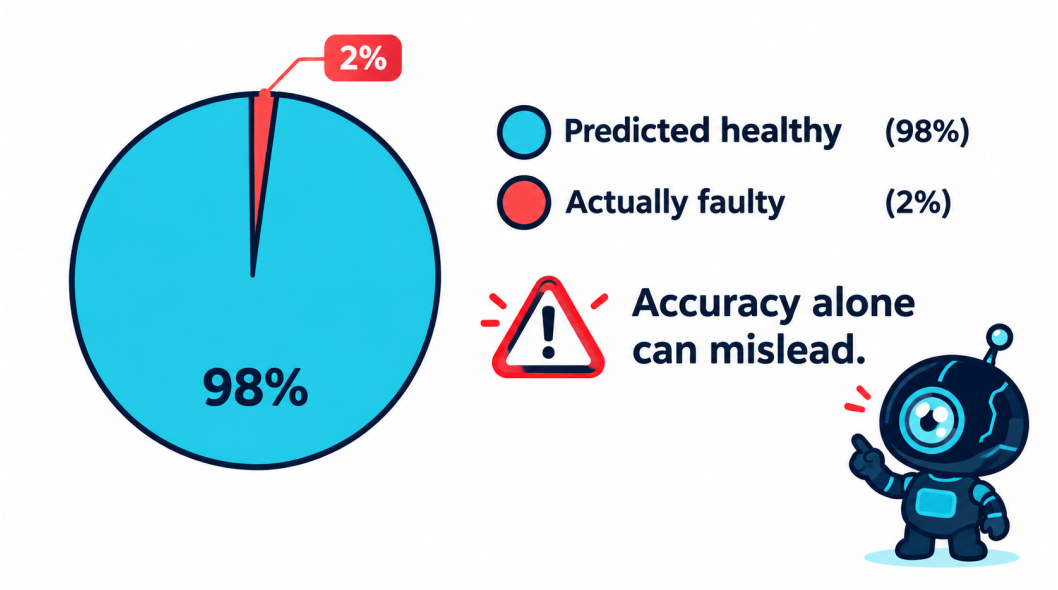



*Points on the wrong side of the boundary are misclassified — expected on real data.*

The goal during training isn't a perfect boundary; it's the boundary that gets the fewest cases wrong, or that balances the type of mistake you can least afford (missing a genuine fault is usually far worse than a false alarm).

## Measuring how good a classification model actually is

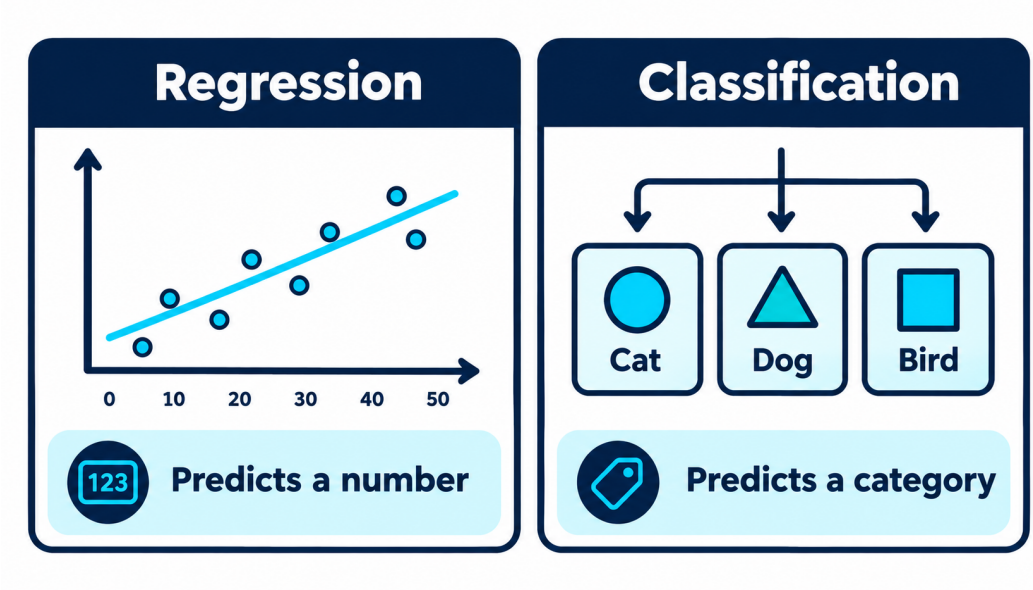
Because the output is a category, not a number, classification models get graded differently from regression models. The simplest measure is accuracy — what percentage of predictions were correct. But accuracy alone can be misleading:




*98% accuracy looks great — until you see what it's actually measuring.*

if only 2% of parts in a dataset are actually faulty, a model that just predicts "healthy" every single time would still be 98% accurate, while being completely useless at its actual job. You'll get into sharper ways of measuring classification performance once you're building models directly — for now, the concept to hold onto is that raw accuracy doesn't tell the whole story, especially when one category is much rarer than the other.

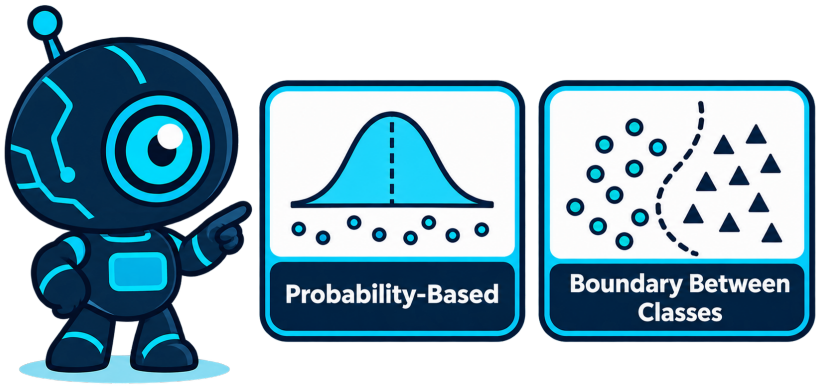
## Where classification fits next to regression




*Regression predicts a number; classification predicts a category.*

## Where this leaves you

By the end of this lesson, you should be able to recognize a classification problem on sight — the target is one of a fixed set of categories, whether that's pass/fail, healthy/worn, or a specific fault type — and understand what's happening underneath: the model assigns a probability to each category and finds the boundary that best separates them, knowing that boundary will rarely be perfect on real data. With regression and classification both covered, you now have the two core tools of supervised learning. Next, you bring them together in a hands-on capstone project, where you'll build a model that predicts a real outcome from real data.




*The two core ideas: probability-based, and a boundary between classes.*